

Практическая работа №1

Теоретическое введение

Docker — это открытая платформа для разработки, доставки и запуска приложений в контейнерах. Контейнеризация — это технология, которая позволяет упаковывать приложение и все его зависимости в единый стандартизированный блок, называемый контейнером. Контейнеры обеспечивают изоляцию приложений и их зависимостей, что позволяет приложениям запускаться в одинаковом окружении независимо от того, где они развернуты.

Основные концепции Docker включают в себя:

1. Образы (Images): Образ — это шаблон для создания контейнера. Он содержит все необходимое для запуска приложения, включая код, среду выполнения, библиотеки, зависимости и настройки. Образы могут быть созданы вручную или автоматически с помощью Dockerfile, который содержит инструкции для сборки образа.

2. Контейнеры (Containers): Контейнер — это инстанс образа. Когда вы запускаете контейнер, Docker использует образ как основу и создает экземпляр контейнера. Контейнер работает в изолированном окружении, где у него есть свое собственное файловое пространство, сеть и процессы.

3. Dockerfile: Dockerfile — это текстовый файл, содержащий инструкции для сборки Docker образа. Он определяет, какие команды и операции нужно выполнить, чтобы создать образ приложения. Dockerfile включает в себя шаги по установке зависимостей, копированию файлов, настройке окружения и многое другое.

4. Docker Hub: Docker Hub — это облачное хранилище для образов Docker. Здесь вы можете найти тысячи общедоступных образов, которые можно использовать в своих проектах. Также можно загружать свои собственные образы на Docker Hub и делиться ими с другими.

5. CLI (Command Line Interface): Docker CLI — это интерфейс командной строки для управления Docker. С его помощью вы можете создавать, запускать, останавливать и удалять контейнеры, собирать образы, работать с сетями и томами, а также многое другое.

Docker стал популярным инструментом в сфере разработки ПО и DevOps благодаря своей простоте использования, эффективности и переносимости. Он позволяет разработчикам быстро создавать, тестировать

и развертывать приложения в любом окружении, что делает процесс разработки и развертывания приложений более гибким и эффективным.

Полезные ссылки

1. Официальная документация Docker: <https://docs.docker.com/>
2. Docker Hub: <https://hub.docker.com/>
3. Docker Community Forums: <https://forums.docker.com/>

Задание

Данная практическая работа представляет собой набор задач, направленных на знакомство и работу с Docker. Docker — это популярная платформа для разработки, доставки и запуска приложений в контейнерах. Контейнеры обеспечивают изоляцию приложений и их зависимостей, что делает процесс разработки и развертывания приложений более простым и надежным.

Практическая работа включает в себя следующие шаги:

Запуск Docker: убедитесь, что у вас установлен Docker на вашем компьютере или сервере, и Docker сервис запущен.

Проверка работоспособности на контейнере hello world: это первоначальная проверка, чтобы убедиться, что Docker работает правильно на вашей системе.

Использование контейнера PostgreSQL с Docker Hub: Скачайте и запустите образ PostgreSQL из Docker Hub. Это даст вам опыт работы с образами, создание и управление контейнерами, а также настройку переменных окружения для настройки контейнера.

Создание своего контейнера и его запуск: Создайте свой собственный контейнер, используя Dockerfile, и убедитесь, что он успешно запускается. В данном случае мы создаем простой контейнер с веб-сервером Nginx.

Базовые команды работы с Docker: изучите базовые команды Docker для управления контейнерами, включая просмотр, запуск, остановку и удаление контейнеров. Это поможет вам эффективно управлять вашими контейнерами в будущем.

Эти шаги позволят вам ознакомиться с основами работы с Docker и использовать его для разработки, тестирования и развертывания ваших приложений. Практическое выполнение этих задач поможет улучшить ваши навыки в области контейнеризации и управления приложениями.

Установка

Для начала необходимо установить Docker. Для установки Docker на Fedora необходимо воспользоваться следующими командами:

Удаление старых версий Docker:

```
sudo dnf remove docker \
                    docker-client \
                    docker-client-latest \
                    docker-common \
                    docker-latest \
                    docker-latest-logrotate \
                    docker-logrotate \
                    docker-selinux \
                    docker-engine-selinux \
                    docker-engine
```

Далее необходимо установить пакет `dnf-plugins-core` (который предоставляет команды для управления вашими репозиториями DNF) и настроить репозиторий:

```
sudo dnf -y install dnf-plugins-core
sudo dnf config-manager --add-repo
https://download.docker.com/linux/fedora/docker-
ce.repo
```

Установите Docker Engine, Containerd и Docker Compose:

```
sudo dnf install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

Запуск Docker:

```
sudo systemctl start docker
```

Убедитесь, что установка Docker Engine прошла успешно, запустив образ `hello-world`:

```
sudo docker run hello-world
```

Использование контейнера postgresql с docker hub

Скачиваем образ Postgres из Docker Hub:

```
docker pull postgres
```

Запускаем контейнер, указывая имя, пароль и создаваемую базу данных:

```
docker run --name имя -e POSTGRES_PASSWORD=пароль -d  
postgres
```

Создаем базу данных:

```
docker exec -it postgres-container psql -U postgres -c  
"CREATE DATABASE mydatabase;"
```

Создание собственного контейнера:

Для начала создание собственного контейнера желательно создать отдельную директорию для дальнейшей работы. В ней необходимо создать Dockerfile.

Пример содержимого Docker-файла:

```
FROM ubuntu:latest  
RUN apt-get update && apt-get install -y nginx  
CMD ["nginx", "-g", "daemon off;"]
```

Сборка контейнера:

```
docker build -t my-nginx-container .
```

Запуск контейнера:

```
docker run --name my-nginx -d -p 80:80 my-nginx-  
container
```

Базовые команды для работы с Docker

Просмотр всех контейнеров:

```
docker ps -a
```

Просмотр запущенных контейнеров:

```
docker ps
```

Остановка контейнера:

```
docker stop <container_name_or_id>
```

Запуск остановленного контейнера:

```
docker start <container_name_or_id>
```

Удаление контейнера:

```
docker rm <container_name_or_id>
```

Проверка статуса контейнера:

```
docker inspect <container_name_or_id>
```

Вопросы к практической работе

1. Что такое Docker и какие преимущества он предоставляет для разработки и развертывания приложений?
2. Какие основные концепции Docker вы знаете, и как они взаимодействуют между собой?
3. Какие шаги необходимо выполнить для установки Docker на локальном компьютере или сервере?
4. Какие команды Docker вы знаете для работы с контейнерами (создание, запуск, остановка, удаление)?
5. Что такое Dockerfile, и какие инструкции он может содержать?

6. Какие основные различия между образом (image) и контейнером (container) в Docker?
7. Как можно использовать переменные окружения в Docker контейнерах, и зачем это нужно?
8. Какие способы доступа к контейнерам Docker извне (например, открытие портов)?
9. Как можно объединить несколько контейнеров в один проект с помощью Docker Compose?
10. Какие полезные ресурсы и сообщества поддержки существуют для пользователей Docker, и как ими воспользоваться для получения помощи или дополнительного обучения?

Критерии оценки

- Установлен Docker
- Запущен Docker
- Использован контейнер PostgreSQL с Docker Hub:
- Создан и запущен собственный контейнер:
- Проработаны базовые команды работы с Docker
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе